



APB PROTOCOL

ADVANCED PERIPHERAL BUS PROTOCOL

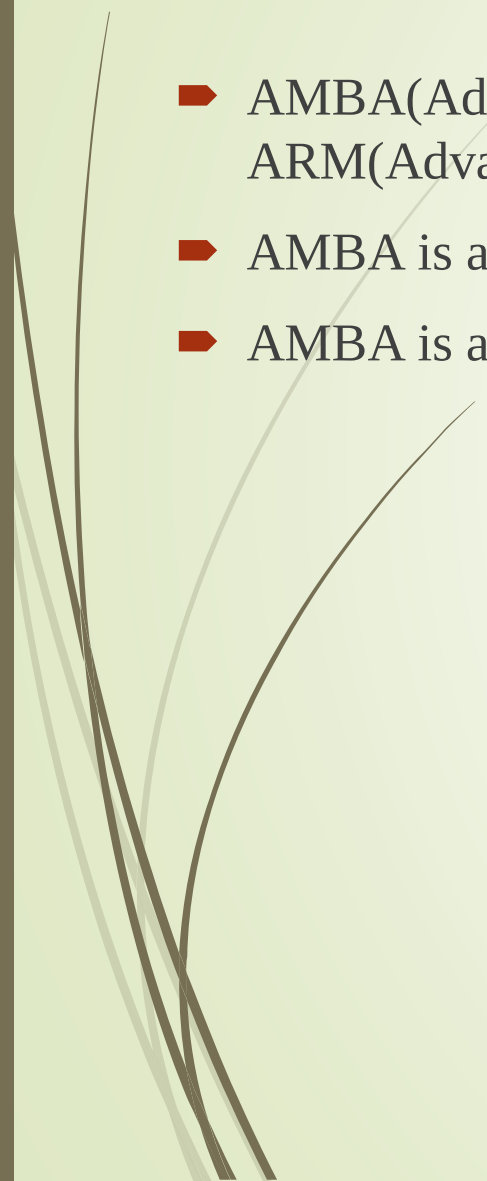


INDEX

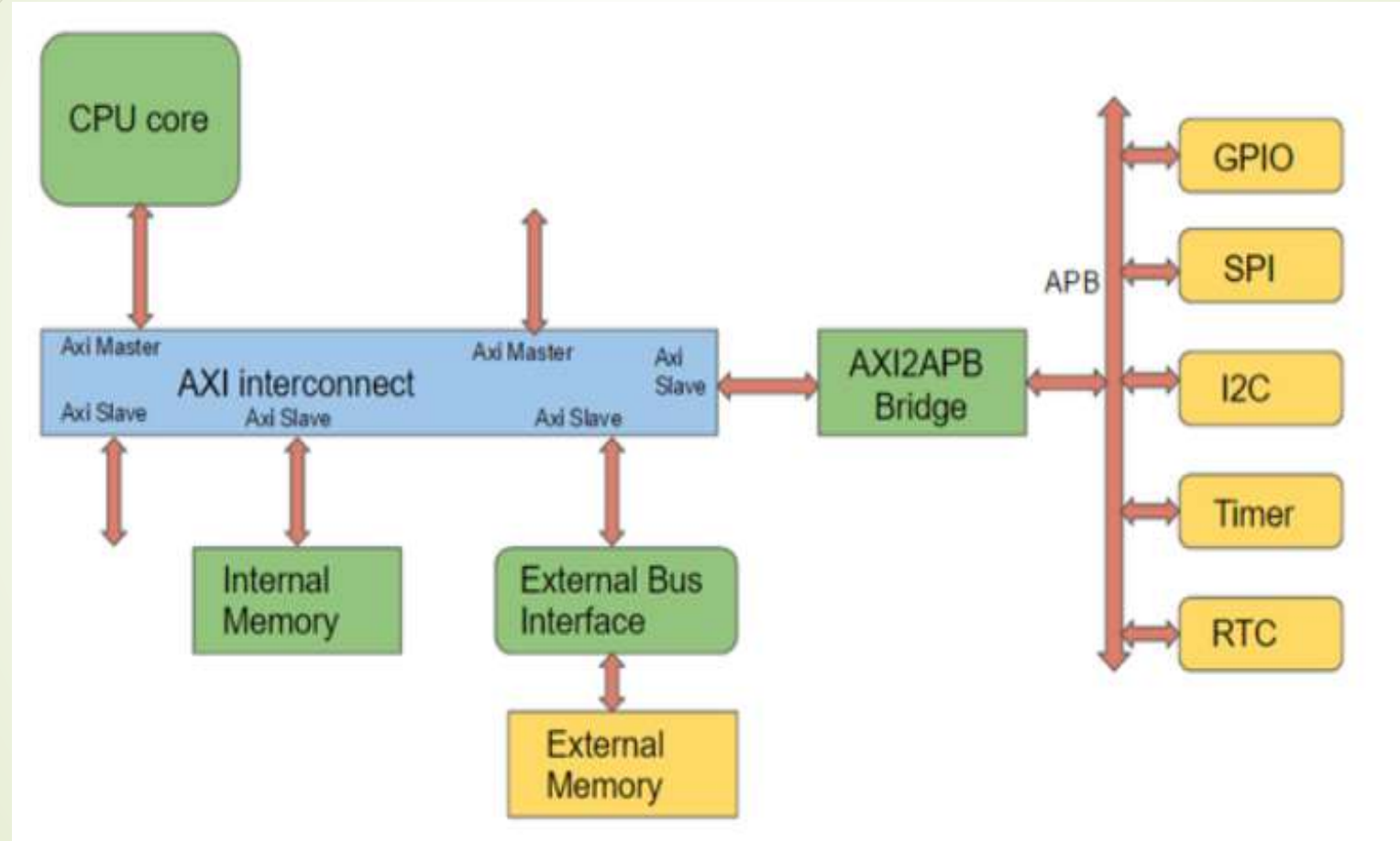
- INTRODUCTION OF AMBA
- ABOUT AMBA BUS ARCHITECTURE
- INTRODUCTION OF APB PROTOCOL
- SIGNAL DESCRIPTIONS
- MASTER- SLAVE COMMUNICATION
- STATE MACHINE
- TIMING DIAGRAMS



INTRODUCTION OF AMBA

- AMBA(Advanced micro control Bus Architecture) is introduced by ARM(Advanced RISC Machine) in 1996.
 - AMBA is a family for all protocols. Like as ASB,APB,AHB, AXI....,etc.
 - AMBA is a set of interconnect protocols.
- 

AMBA BUS ARCHITECTURE





APB PROTOCOL

- APB protocol is a part of AMBA family. APB protocol is low-cost interface that optimized for minimal power consumption.
- It is reduced interface complexity. It is not pipelined and is a simple.
- Every transfer takes at least two cycles to complete.
- APB Bridge connected from AHB.
- All the singles transitions are only related to the raising edge of the clock to enable the integration of APB peripherals.

Signal Descriptions

- **PCLK :** All APB signals are timed against the rising edge of PCLK.
- **PRESETn :** PRESETn is the reset signal and is Active-LOW.
PRESETn is normally connected directly to the system bus reset signal.
- **PADDR :** PADDR is the APB address bus. PADDR can be up to 32 bits wide.
- **PSELx :** The **APB** bridge unit generates this signal to each peripheral bus slave. It indicates that the slave is selected and that a data transfer is required. There is each **PSELx** signal for each slave.
- **PENABLE:** **PENABLE** indicates the second and subsequent cycles of an APB transfer.
- **PWRITE:** The signal indicates an **APB** write access when **HIGH** and an **APB** read access when **LOW**.
- **PWDATA:** The PWDATA write data bus is driven by the APB bridge when **PWRITE** is **HIGH**.

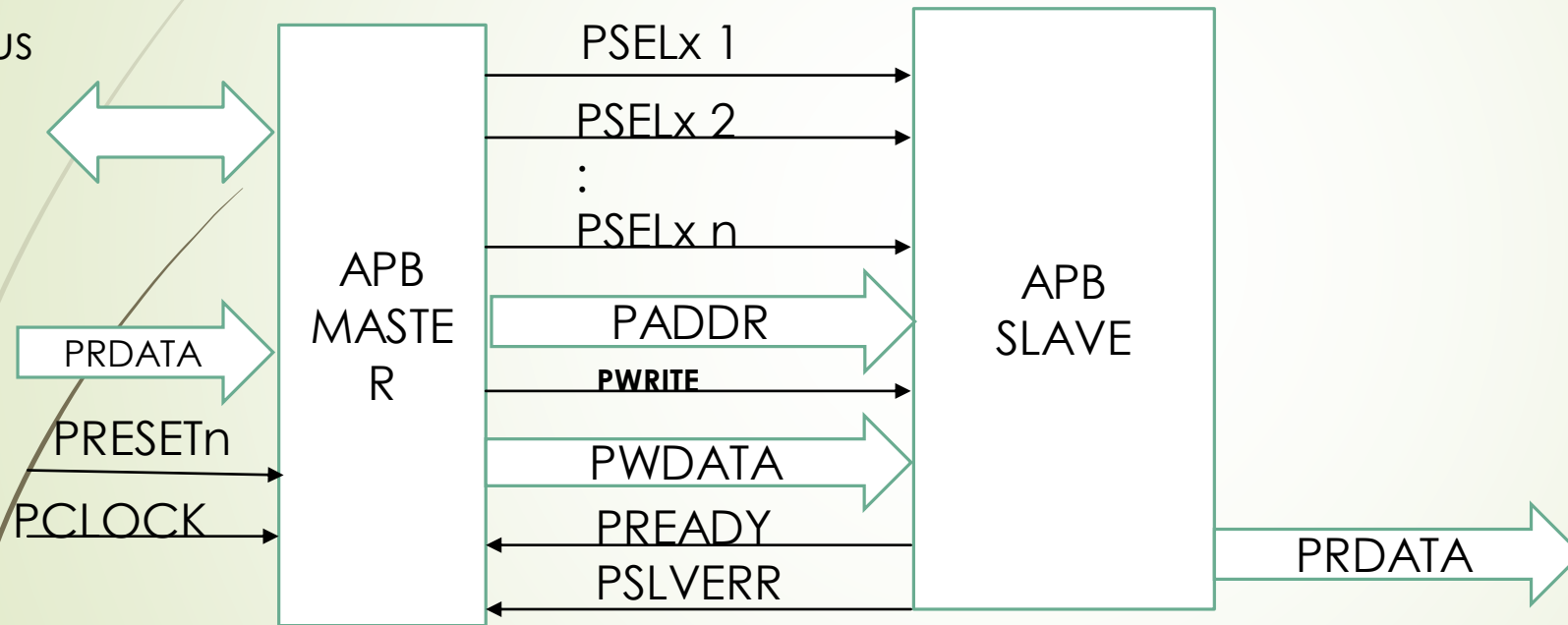


SIGNAL DESCRIPTIONS..

- **PRDATA** : The **PRDATA** read data bus is driven by the APB peripheral bus slave when **PWRITE** is **LOW**
- **PREADY**: The APB peripheral slave uses this signal to extend an APB transfer.
- **PSLVERR**: This signal indicates transfer failure. **PSLVERR** is **HIGH** when **PSELx**, **PENABLE**, **PREADY** are **HIGH** else becomes **LOW**.

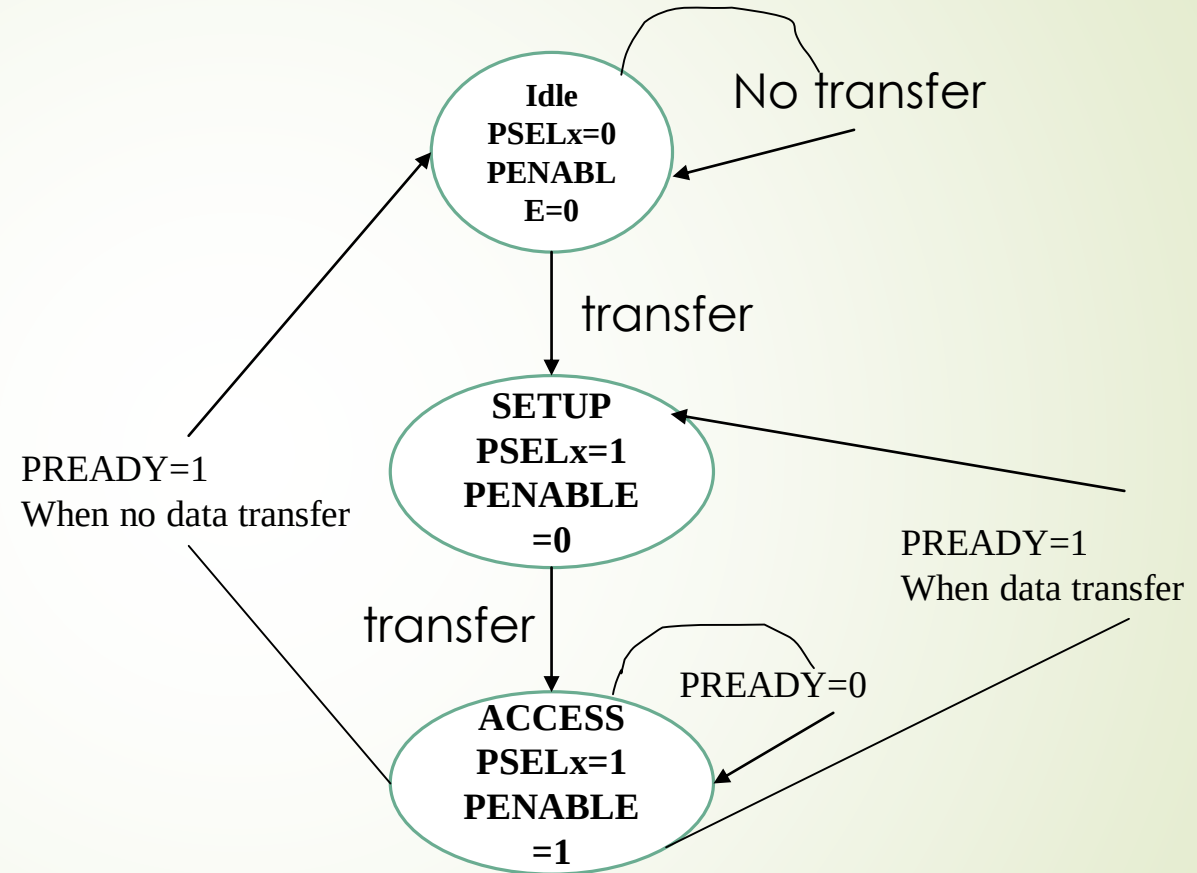
MASTER- SLAVE COMMUNICATION:

System Bus
slave
interface



STATE MACHINE:

- IDLE
- Default APB State
- **SETUP**
- When Transfer required
- PSELx=1 asserted
- Only one cycle
- **ACCESS**
- PENABLE is asserted, where Address, write, select, and write data remain same.
- Stay if PREADY=0,
- Else go to IDLE if PREADY=1
- No data transfer.
- go to SETUP is PREADY=1, when more data pending.



WRITE TRANSFERS:

This section describes the following types of write transfer:

- With no wait states
- With wait states

All signals shown in this section are sampled at the rising edge of PCLK.

With no wait states:

The Setup phase of the write transfer occurs at T1 in Figure.

The select signal, **PSELx**, is asserted, which means that **PADDR**, **PWRITE** and **PWDATA** must be valid.

The Access phase of the write transfer is shown at T2 in Figure. where **PENABLE** is asserted.

PREADY is asserted by the Completer at the rising edge of **PCLK** to indicate that the write data will be accepted at T3. **PADDR**, **PWDATA**, and any other control signals, must be stable until the transfer completes. At the end of the transfer, **PENABLE** is deasserted. **PSELx** is also deasserted, unless there is another transfer to the same peripheral.

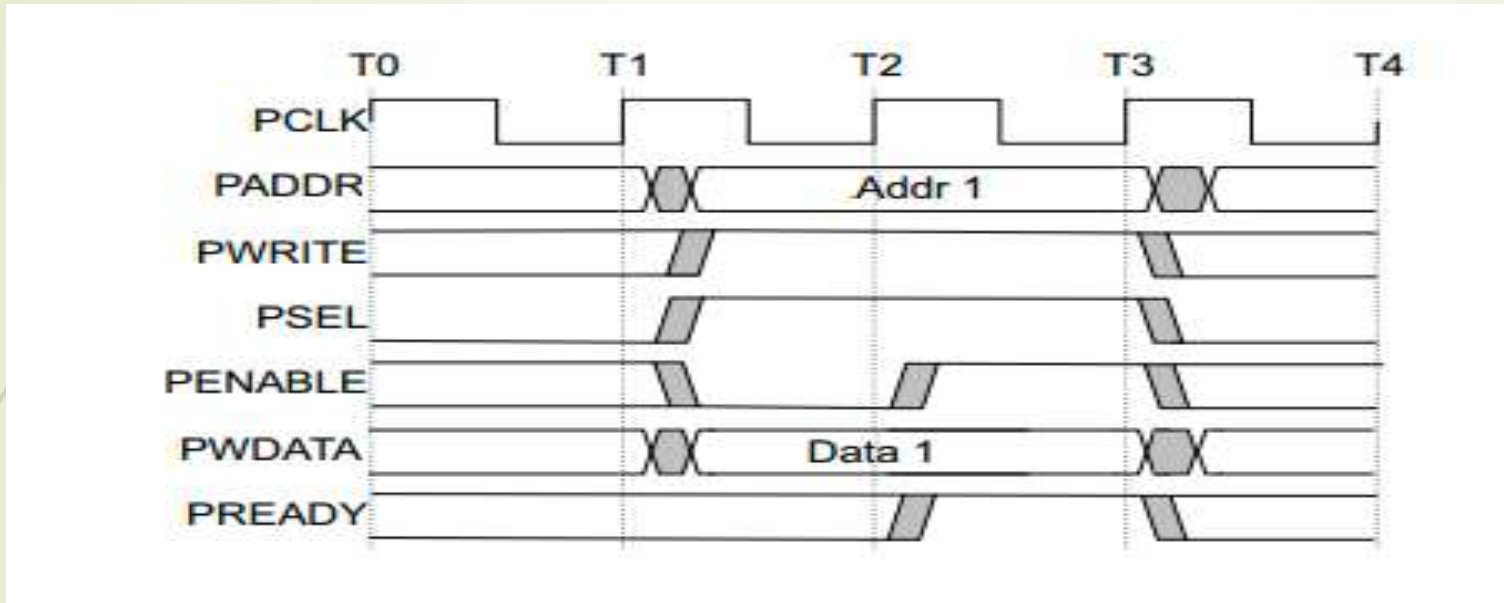


Figure: Write transfer no waits states.

With wait state:

During an Access phase, when **PENABLE** is **HIGH**, the APB peripheral extends the transfer by driving **PREADY LOW**. The following signals remain unchanged while **PREADY** remains **LOW**:

- Address signal, **PADDR**
- Direction signal, **PWRITE**
- Select signal, **PSELx**
- Enable signal, **PENABLE**
- Write data signal, **PWDATA**
- **PREADY** can take any value when **PENABLE** is **LOW**. This ensures that peripherals that have a fixed two cycle access can tie **PREADY HIGH**.

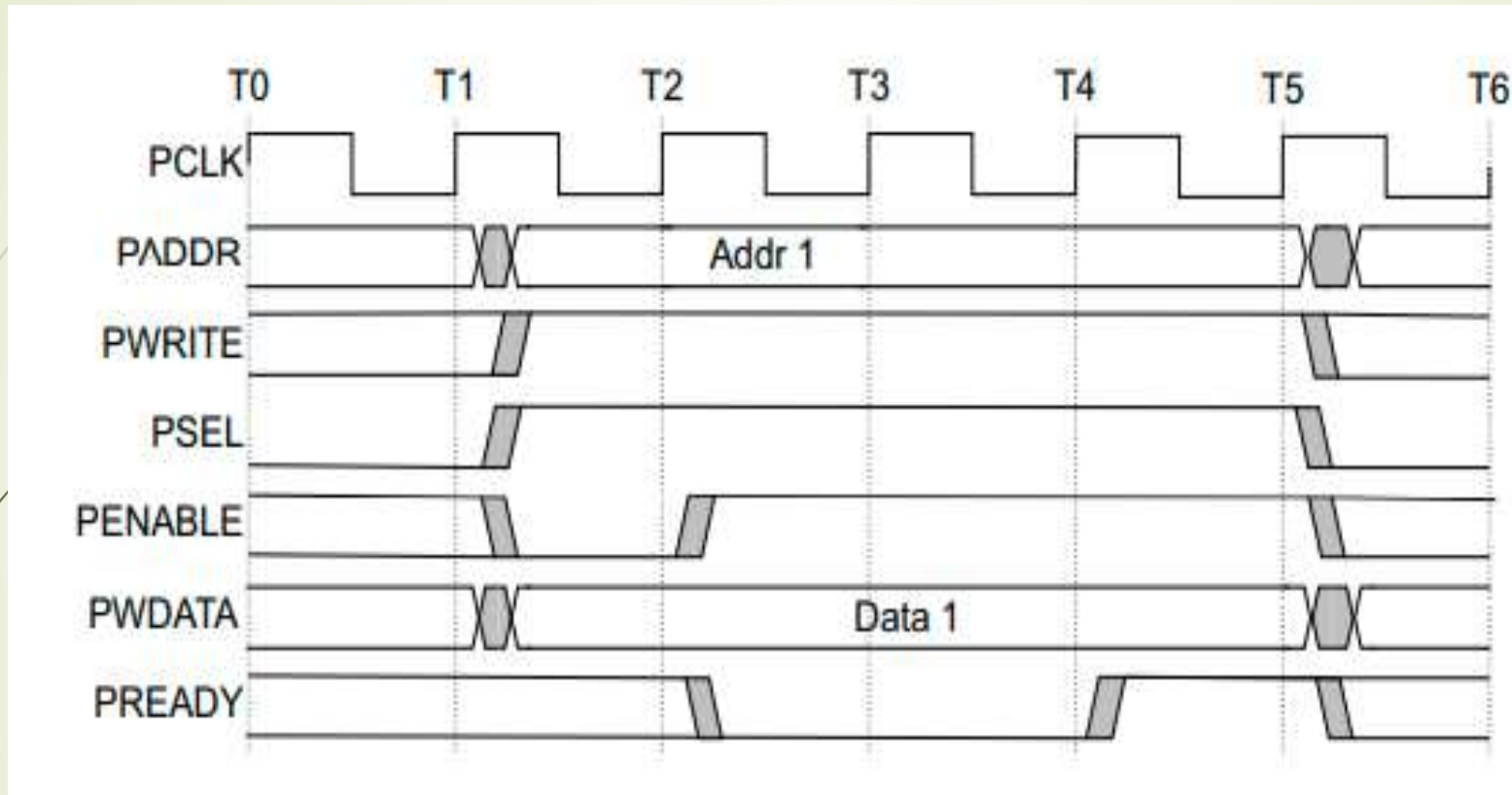


Figure: With wait state transfer

READ TRANSFER:

Two types of read transfer are described in this section:

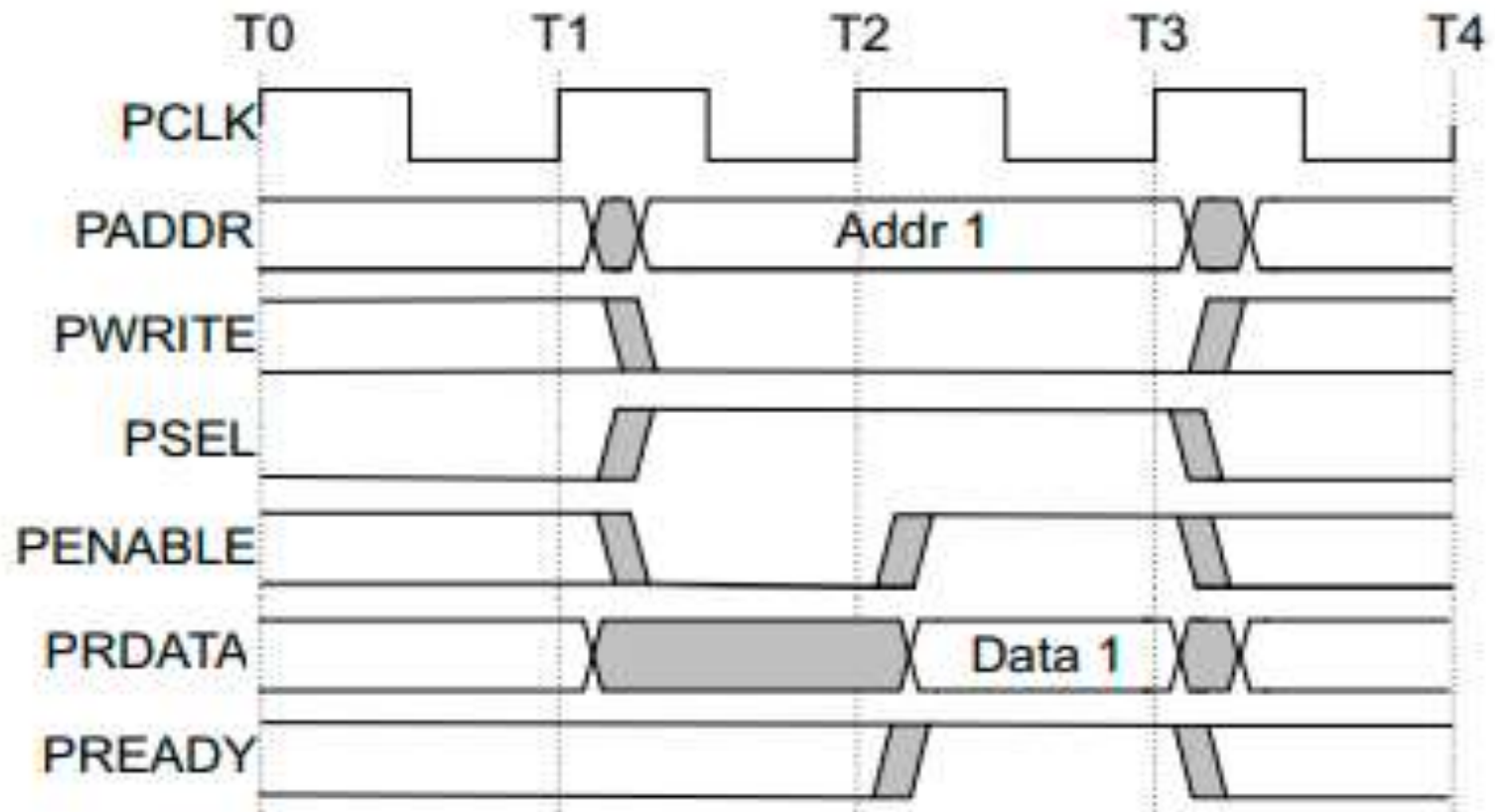
- With no wait states
- With wait states

All signals shown in this section are sampled at the rising edge of **PCLK**.

With no wait states:

The timing of the address, **PADDR**, write, **PWRITE**, select, **PSELx**, and enable, **PENABLE**, signals are the same as described in Write transfers on page.

The Completer must provide the data before the end of the read transfer.

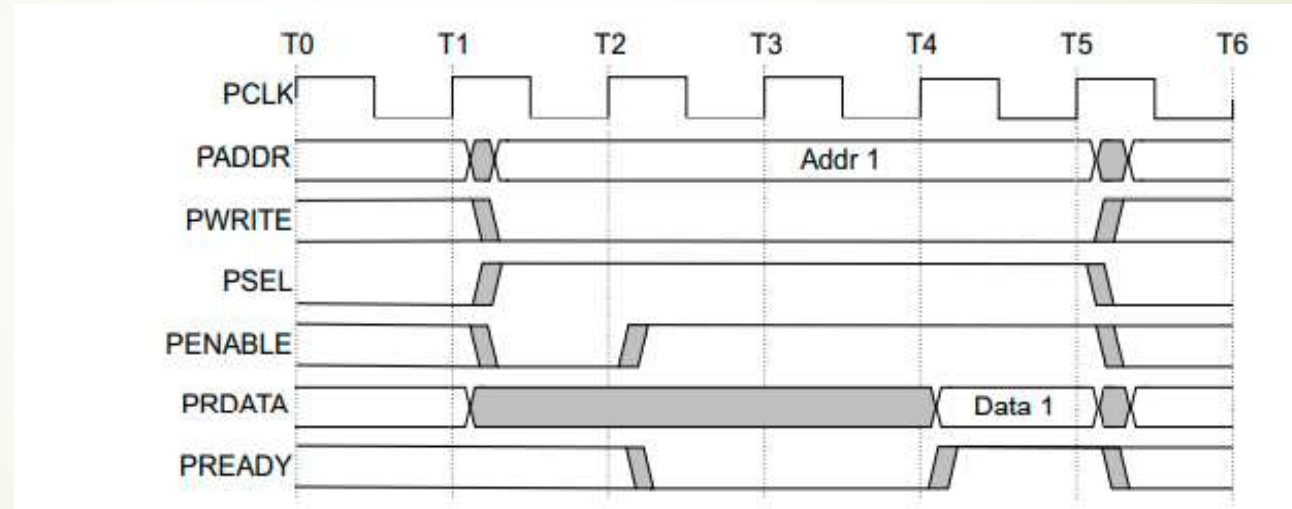


Read transfer with no wait states

With wait states:

The transfer is extended if **PREADY** is driven **LOW** during an Access phase. The following signals remain unchanged while **PREADY** remains **LOW**:

- Address signal, **PADDR**
- Direction signal, **PWRITE**
- Select signal, **PSEL**
- Enable signal, **PENABLE**



Read transfer with wait states

Figure: Shows that two cycles are added using **PREADY**. However, any number of additional cycles can be added, from zero upwards.

Error response:

PSLVERR can be used to indicate an error condition on an **APB** transfer. Error conditions can occur on both read and write transactions.

PSLVERR is only considered valid during the last cycle of an **APB** transfer, when **PSEL**, **PENABLE**, and **PREADY** are all **HIGH**.

It is recommended, but not required, that **PSLVERR** is driven **LOW** when **PSEL**, **PENABLE**, or **PREADY** are **LOW**.

Transactions that receive an error might or might not have changed the state of the peripheral. This is peripheral-specific and either state is acceptable.

When a write transaction receives an error, this does not mean that the register within the peripheral has not been updated.

Read transactions that receive an error can return invalid data. There is no requirement for the peripheral to drive the data bus to all 0s for a read error.

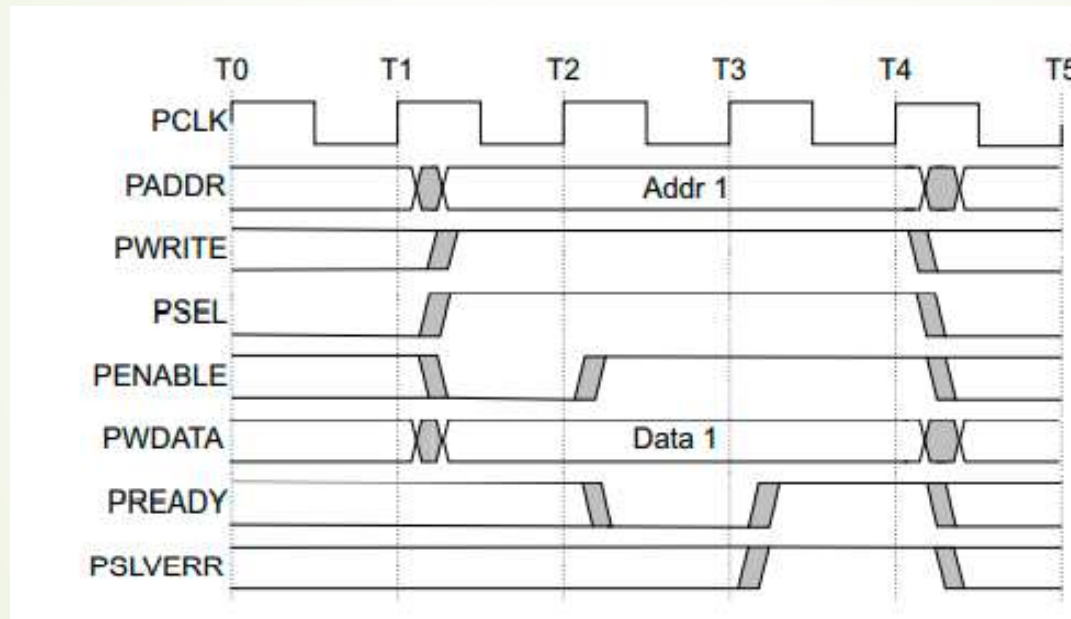
A Requester which receives an error response to a read transfer might still use the data.

A Completer cannot rely on the error response to prevent the reading of a value on **PRDATA**.

Completers are not required to support **PSLVERR**. Where a Peripheral does not include **PSLVERR**, the appropriate input to the Requester is tied **LOW**.

Write transfer error:

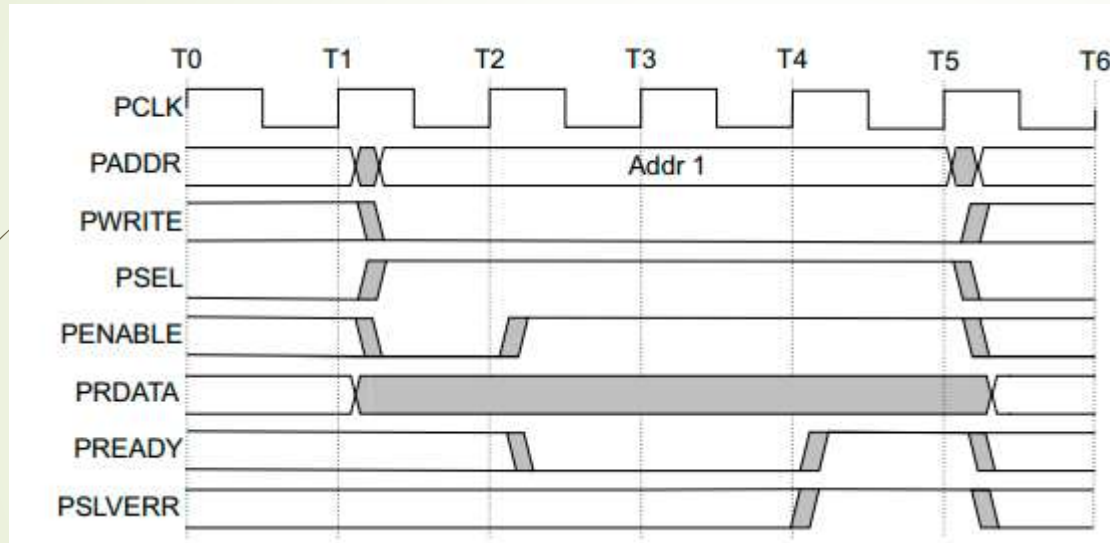
An example of a failing write transfer that completes with an error.



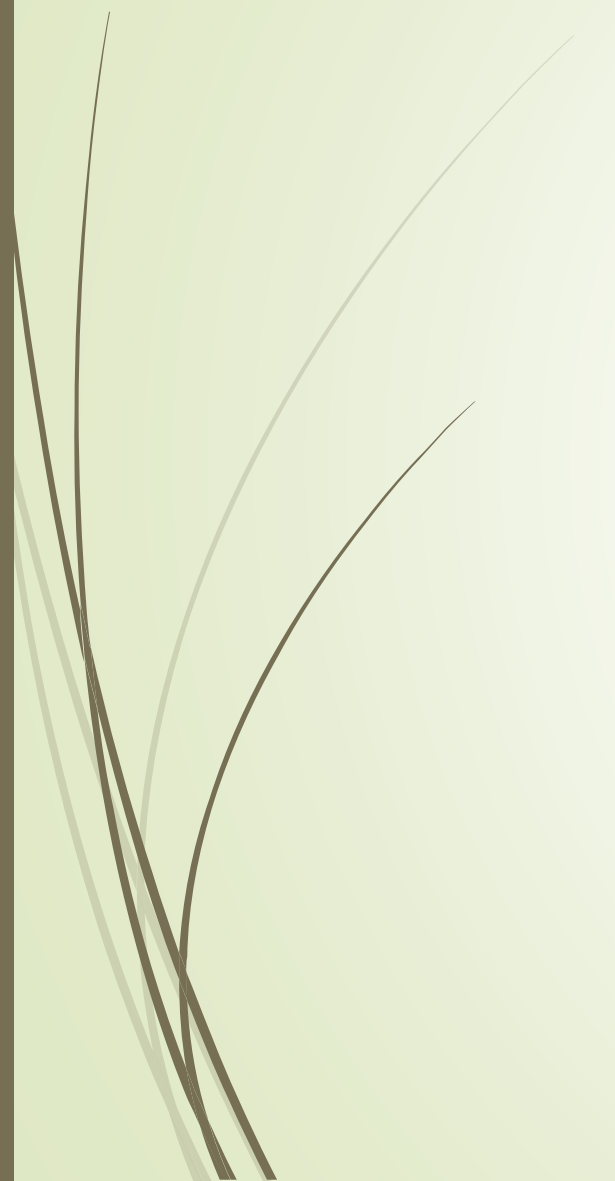
Example of failing write transfer

Read transfer error:

A read transfer can also complete with an error response, indicating that there is no valid read data available.



Example failing read transfer



*Thank
you !!!*